

IMDB Movie Review Sentiment Classifier

(Data Storage, Data Preparation and Reproducibility Documentation)

Student Name: Omprakash Vootla

Student ID: W20122718

Email: 20122718@setu.ie-

Project Overview:

The notebook prepares the IMDB movie-review dataset for a downstream binary sentiment-classification system. The task is to use a movie review as input and classify its sentiment as either positive or negative. The notebook focuses on cloud storage, data validation, reproducible train/dev/test splitting, stratified cross-validation, and creation of CSV datasets in Google Cloud Storage (GCS).

- Dataset: IMDB 50,000 movie reviews.
- Target: sentiment, with two classes: positive and negative.
- Raw data location: Google Cloud Storage bucket gs://imdb-sentiment-project/raw/IMDB Dataset.csv.
- Main split: 80% train, 10% development, 10% test.
- Cross-validation: 8-fold stratified CV generated from the training set only.
- Output: 19 derived CSV files — train, dev, test, plus 16 CV train/validation files.

Criterion 1: Raw data storage

The raw IMDB dataset is stored in Google Cloud Storage (GCS), which is used as the project's object-storage layer.

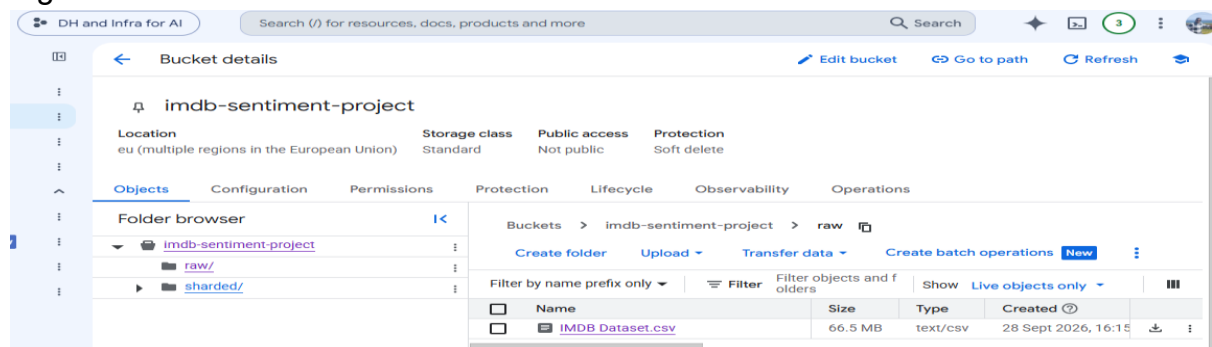
The notebook explicitly configures as,

PROJECT_ID = "dh-and-infra-for-ai",

BUCKET_NAME = "imdb-sentiment-project",

and RAW_GCS_PATH = "raw/IMDB Dataset.csv".

Figure 1:



The resulting raw object is therefore:
gs://imdb-sentiment-project/raw/IMDB Dataset.csv

The notebook downloads this object to the Colab working environment as "IMDB Dataset.csv". The raw object is deliberately kept separate from the derived datasets and is not overwritten after splitting. This provides a stable source from which the processing workflow can be rerun.

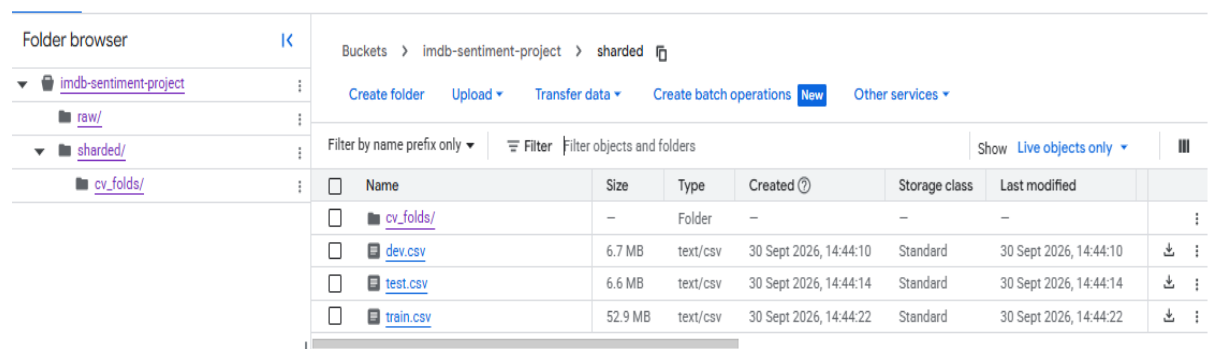
Criterion 2: Processed data storage & file formats

Processed/derived data is stored back in the same GCS bucket under the "sharded/" prefix. The notebook creates CSV files for the main partitions and the cross - validation folds.

The structure is:

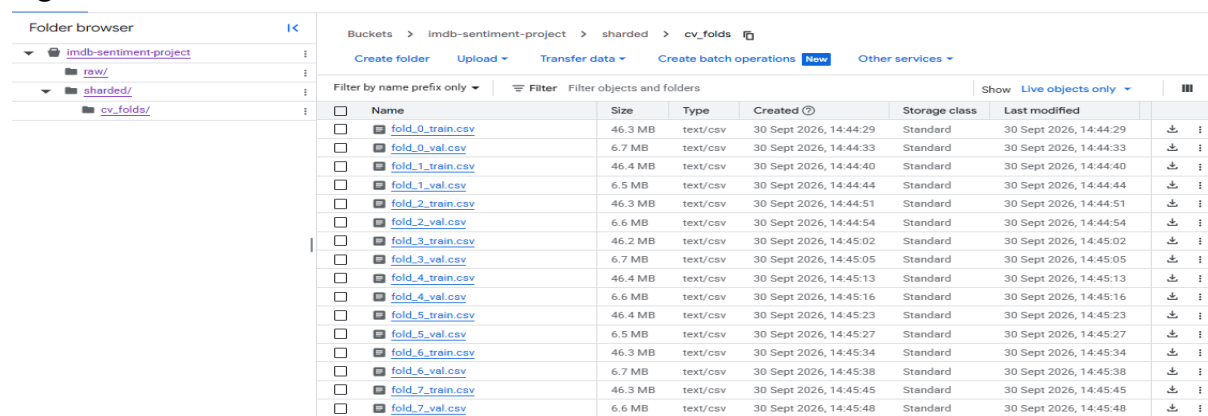
```
gs://imdb-sentiment-project/ raw/IMDB Dataset.csv
sharded/train.csv
sharded/dev.csv
sharded/test.csv
sharded/cv_folds/fold_0_train.csv
sharded/cv_folds/fold_0_val.csv ...
sharded/cv_folds/fold_7_train.csv
sharded/cv_folds/fold_7_val.csv
```

Figure 2:



Name	Size	Type	Created	Storage class	Last modified
cv_folds/	—	Folder	—	—	—
dev.csv	6.7 MB	text/csv	30 Sept 2026, 14:44:10	Standard	30 Sept 2026, 14:44:10
test.csv	6.6 MB	text/csv	30 Sept 2026, 14:44:14	Standard	30 Sept 2026, 14:44:14
train.csv	52.9 MB	text/csv	30 Sept 2026, 14:44:22	Standard	30 Sept 2026, 14:44:22

Figure 3:



Name	Size	Type	Created	Storage class	Last modified
fold_0_train.csv	46.3 MB	text/csv	30 Sept 2026, 14:44:29	Standard	30 Sept 2026, 14:44:29
fold_0_val.csv	6.7 MB	text/csv	30 Sept 2026, 14:44:33	Standard	30 Sept 2026, 14:44:33
fold_1_train.csv	46.4 MB	text/csv	30 Sept 2026, 14:44:40	Standard	30 Sept 2026, 14:44:40
fold_1_val.csv	6.5 MB	text/csv	30 Sept 2026, 14:44:44	Standard	30 Sept 2026, 14:44:44
fold_2_train.csv	46.3 MB	text/csv	30 Sept 2026, 14:44:51	Standard	30 Sept 2026, 14:44:51
fold_2_val.csv	6.6 MB	text/csv	30 Sept 2026, 14:44:54	Standard	30 Sept 2026, 14:44:54
fold_3_train.csv	46.2 MB	text/csv	30 Sept 2026, 14:45:02	Standard	30 Sept 2026, 14:45:02
fold_3_val.csv	6.7 MB	text/csv	30 Sept 2026, 14:45:05	Standard	30 Sept 2026, 14:45:05
fold_4_train.csv	46.4 MB	text/csv	30 Sept 2026, 14:45:13	Standard	30 Sept 2026, 14:45:13
fold_4_val.csv	6.6 MB	text/csv	30 Sept 2026, 14:45:16	Standard	30 Sept 2026, 14:45:16
fold_5_train.csv	46.4 MB	text/csv	30 Sept 2026, 14:45:23	Standard	30 Sept 2026, 14:45:23
fold_5_val.csv	6.5 MB	text/csv	30 Sept 2026, 14:45:27	Standard	30 Sept 2026, 14:45:27
fold_6_train.csv	46.3 MB	text/csv	30 Sept 2026, 14:45:34	Standard	30 Sept 2026, 14:45:34
fold_6_val.csv	6.7 MB	text/csv	30 Sept 2026, 14:45:38	Standard	30 Sept 2026, 14:45:38
fold_7_train.csv	46.3 MB	text/csv	30 Sept 2026, 14:45:45	Standard	30 Sept 2026, 14:45:45
fold_7_val.csv	6.6 MB	text/csv	30 Sept 2026, 14:45:48	Standard	30 Sept 2026, 14:45:48

The notebook saves the pandas Data Frames with `DataFrame.to_csv`. CSV is appropriate here because the dataset contains two simple tabular fields: review text and a categorical sentiment label, and the generated files need to be easy to inspect and consume in Python.

Criterion 3: Database / object storage decision

The implementation chooses object storage rather than a relational database. Google Cloud Storage is suitable because the project is storing relatively large immutable/semi-static files rather than performing transactional database operations.

The data is naturally represented as files containing review text and labels, therefore does not require SQL queries, joins, transactions, or relational database constraints.

Criterion 4: Data versioning

The notebook provides some version-control characteristics, although it does not implement a formal dataset-versioning system.

First, the raw dataset is stored separately under "raw/" and the notebook states that it is not overwritten after the train/dev/test split. Derived data is stored separately under "sharded/". However, there is no explicit version identifier such as v1.0, a timestamped dataset folder, Git commit identifier, checksum manifest, or GCS object-generation record written by the notebook.

Criterion 5: Data access

The GCS notebook uses Google Colab authentication through `google.colab.auth.authenticate_user()`.

It then creates a `google.cloud.storage Client` using the configured project and accesses the `imdb-sentiment-project` bucket.

Raw access: `bucket.blob("raw/IMDB Dataset.csv")` followed by `download_to_filename("IMDB Dataset.csv")`.

Processed upload: generated local CSV files are uploaded with `blob.upload_from_filename(...)`.

Bucket verification: `storage_client.list_blobs(BUCKET_NAME)` lists the stored objects.

Direct model-data access: pandas read a GCS URI such as `gs://imdb-sentiment-project/sharded/cv_folds/fold_0_val.csv`.

The second notebook reads `gs://imdb-sentiment-project/sharded/train.csv` directly with `pd.read_csv()`.

Authentication is session-based in Colab; credentials are not hard-coded into the notebook.

Criterion 6: Data split / validation strategy

The primary partitioning strategy in the GCS notebook is an 80% training, 10% development and 10% test split. The split is stratified on sentiment.

First split: `train_test_split(df, test_size=0.20, random_state=42, stratify=df["sentiment"])`.

Second split: the 20% temporary set is divided equally into development and test using `test_size=0.50, random_state=42` and `stratify=temp_df["sentiment"]`.

Result: 40,000 train rows, 5,000 development rows and 5,000 test rows.

Figure 4:

```
RANDOM_STATE = 42

# First split: 80% train, 20% temporary
train_df, temp_df = train_test_split(
    df,
    test_size=0.20,
    random_state=RANDOM_STATE,
    stratify=df["sentiment"]
)

# Second split: divide the 20% temporary set equally into
# 10% development and 10% test.
dev_df, test_df = train_test_split(
    temp_df,
    test_size=0.50,
    random_state=RANDOM_STATE,
    stratify=temp_df["sentiment"]
)

print(f"Train: {len(train_df):,} rows ({len(train_df)/len(df):.1%}")
print(f"Dev:   {len(dev_df):,} rows ({len(dev_df)/len(df):.1%}")
print(f"Test:  {len(test_df):,} rows ({len(test_df)/len(df):.1%}")
```

```
Train: 40,000 rows (80.0%)
Dev:   5,000 rows (10.0%)
Test:  5,000 rows (10.0%)
```

Figure 5:

```
def show_split_distribution(name, data):
    counts = data["sentiment"].value_counts()
    proportions = data["sentiment"].value_counts(normalize=True)

    print(f"--- {name} ---")
    print(f"Rows: {len(data):,}")
    print("Counts:")
    print(counts.to_dict())
    print("Proportions:")
    print(proportions.round(4).to_dict())
    print()

show_split_distribution("Train", train_df)
show_split_distribution("Dev", dev_df)
show_split_distribution("Test", test_df)
|
```

```
--- Train ---
Rows: 40,000
Counts:
{'positive': 20000, 'negative': 20000}
Proportions:
{'positive': 0.5, 'negative': 0.5}

--- Dev ---
Rows: 5,000
Counts:
{'negative': 2500, 'positive': 2500}
Proportions:
{'negative': 0.5, 'positive': 0.5}

--- Test ---
Rows: 5,000
Counts:
{'negative': 2500, 'positive': 2500}
Proportions:
{'negative': 0.5, 'positive': 0.5}
```

Observed distributions: train = 20,000 positive / 20,000 negative; dev = 2,500 / 2,500; test = 2,500 / 2,500.

Figure 6:

```
skf = StratifiedKFold(  
    n_splits=8,  
    shuffle=True,  
    random_state=RANDOM_STATE  
)  
  
cv_folds = {}  
  
for fold_idx, (train_idx, val_idx) in enumerate(  
    skf.split(train_df, train_df["sentiment"])  
):  
    fold_train = train_df.iloc[train_idx].copy()  
    fold_val = train_df.iloc[val_idx].copy()  
  
    cv_folds[fold_idx] = (fold_train, fold_val)  
  
print(f"Number of CV folds: {len(cv_folds)}")  
  
for fold_idx, (fold_train, fold_val) in cv_folds.items():  
    print(  
        f"Fold {fold_idx}: "  
        f"train={len(fold_train):,}, "  
        f"validation={len(fold_val):,}"  
    )  
  
--- Number of CV folds: 8  
Fold 0: train=35,000, validation=5,000  
Fold 1: train=35,000, validation=5,000  
Fold 2: train=35,000, validation=5,000  
Fold 3: train=35,000, validation=5,000  
Fold 4: train=35,000, validation=5,000  
Fold 5: train=35,000, validation=5,000  
Fold 6: train=35,000, validation=5,000  
Fold 7: train=35,000, validation=5,000
```

8-fold StratifiedKFold is generated from train_df only, with shuffle=True and random_state=42. Each CV fold contains 35,000 training rows and 5,000 validation rows.

Criterion 7: Feature description

The field "review" contains the original natural-language text. The second notebook constructs a derived text feature named "clean_review" by applying deterministic text-cleaning rules.

Field	Role	Description
review	Raw input feature	original movie-review text.
Clean_review	Processed input feature	Review after lowercasing, HTML-tag removal, punctuation/special-character removal and whitespace normalization.
sentiment	Original target	Categorical target with values positive and negative.
label	target	Binary numeric target: positive=1 and negative=0.

Figure 7:

```
df.head()
```

	review	sentiment	label
0	I've been looking for the name of this film fo...	positive	1
1	I picked this DVD up at the Dollar Store. The ...	negative	0
2	This was a movie that I hoped I could suggest ...	negative	0
3	Exceptionally silly actioner with braindead le...	negative	0
4	Follow-up to 1973's "Walking Tall" continues t...	negative	0

Figure 8:

```
df[["review", "clean_review"]].head()
```

	review	clean_review
0	I've been looking for the name of this film fo...	i ve been looking for the name of this film fo...
1	I picked this DVD up at the Dollar Store. The ...	i picked this dvd up at the dollar store the d...
2	This was a movie that I hoped I could suggest ...	this was a movie that i hoped i could suggest ...
3	Exceptionally silly actioner with braindead le...	exceptionally silly actioner with braindead le...
4	Follow-up to 1973's "Walking Tall" continues t...	follow up to s walking tall continues the real...

Criterion 8: Data types and formats

- review: text/string data; stored in CSV as a textual field.
- sentiment: categorical/string data with two values, positive and negative.
- label: binary integer/numeric representation created with {"positive": 1, "negative": 0}.
- clean_review: text/string data produced by the text-cleaning function.
- In the second notebook's df.info() reports 39742 rows, two object columns before the label/clean_review fields are added.

Figure 9:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 39742 entries, 0 to 39999
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   review           39742 non-null  object
1   sentiment        39742 non-null  object
2   label            39742 non-null  int64
3   clean_review     39742 non-null  object
dtypes: int64(1), object(3)
memory usage: 1.5+ MB
```

Criterion 9: Reproducibility of data collection

The notebooks reproducibly access the raw dataset once it has been placed in GCS. The source location, bucket name and object path are fixed in code, and the download operation checks for the object's existence before retrieving it.

The reproducible retrieval sequence is authenticate to Google Cloud in Colab → connect to project/bucket → verify raw/IMDB Dataset.csv exists → download it locally → load it with UTF-8 encoding.

Criterion 10: Reproducibility of preprocessing

Together, the notebooks provide a clear sequence of reproducible preparation operations.

A. Load and validate: The GCS notebook reads the raw CSV using `encoding="utf-8"`, checks that review and sentiment exist, retains only those two columns, checks missing values, counts duplicate reviews and checks class proportions.

B. Partition the raw dataset: The GCS notebook creates the stratified 80/10/10 split using `random_state=42`.

C. Create CV folds: Eight stratified folds are generated from the training set only using `StratifiedKFold(n_splits=8, shuffle=True, random_state=42)`.

D. Persist partitions: `train.csv`, `dev.csv`, `test.csv` and the 16 CV files are written using pandas `to_csv(index=False)` and uploaded to GCS.

E. Load the training shard: The second notebook reads `sharded/train.csv` from GCS. The loaded dataset contains 40,000 rows.

F. Remove duplicate rows: The second notebook calculates `df.duplicated().sum()` and then applies `df.drop_duplicates()`. The output shows 258 duplicate rows removed from the 40,000-row training shard, leaving 39,742 rows.

G. Encode the target: sentiment is mapped to label positive → 1 and negative → 0.

H. Clean review text: `clean_text()` lowercases the text, removes HTML tags with `r"<.*?>"`, replaces non-letter/non-whitespace characters with spaces using `r"^[a-zA-Z\s]"`, collapses repeated whitespace using `r"\s+"` and strips leading/trailing whitespace.

I. Create the modelling split: X is `clean_review` and y is label. The second notebook performs an 80/10/10 stratified split with `random_state=42`, producing 31,793 training, 3,974 development and 3,975 test examples.

```
print("Training:", len(X_train))
print("Development:", len(X_dev))
print("Test:", len(X_test))
```

```
Training: 31793
Development: 3974
Test: 3975
```

Figure 10:

Github : <https://github.com/vootlao/Data-Handling-and-Infrastructure-for-AI/tree/main>